

ОТЧЕТ

о выполнении практической работы №1

Дисциплина «Оптимизация веб-приложений»

Тема: «Пользовательские запросы в веб-приложении: DevTools, HTTP, REST, JSON,
Postman»

Специальность: 09.02.07 Информационные системы и программирование

Квалификация: Разработчик веб и мультимедийных приложений

Выполнил: Чеголин Алексей Михайлович

Студент группы: 4ИСИП-923

Проверил:

Преподаватель: Рыбаков А.С.

Дата: «29 сентября 2026 г.»

Оценка: _____

Теоретическая часть:

DevTools — встроенные инструменты разработчика браузера, открываются клавишей F12. Панель Elements показывает структуру страницы, Console — ошибки и выполняемый код, Network — запросы с кодами ответа, заголовками и временем, Performance — скорость работы, Device mode — адаптивность, Security — сертификаты, Memory — потребление памяти.

Пользовательские запросы — это HTTP-запросы, которые браузер отправляет при работе с веб-приложением. Запрос характеризуется методом, адресом и параметрами, заголовками, телом, временем выполнения и кодом ответа. Уменьшить нагрузку на сеть позволяют сокращение числа обращений, уменьшение объёма передаваемых данных, кэширование, пагинация и фильтрация, а также правильный выбор метода и заголовков.

REST — архитектурный стиль взаимодействия с ресурсами по HTTP: клиент-сервер, отсутствие состояния, единообразный интерфейс, уникальные адреса ресурсов. JSON — текстовый формат обмена данными, поддерживающий объекты, массивы, строки, числа, логические значения и null. Postman — инструмент для работы с API: конструирование и отправка запросов, анализ ответов, сохранение коллекций, автотесты.

Метод	CRUD	Назначение	Пример
GET	Read	получение данных, параметры в адресе, данные не меняются	/users?id=5
POST	Create	создание ресурса, данные в теле запроса	регистрация, заказ
PUT	Update	полное обновление: заменяет весь объект	обновить профиль
PATCH	Update	частичное обновление: меняет только указанные поля	сменить email
DELETE	Delete	удаление ресурса	удалить товар
HEAD	—	только заголовки, без тела ответа	проверить размер и дату
OPTIONS	—	какие методы поддерживает ресурс	предварительный запрос

Коды ответов делятся на группы: 1xx информационные, 2xx успешные (200, 201, 204), 3xx перенаправления (301, 302, 304), 4xx ошибки клиента (400, 401, 403, 404), 5xx ошибки сервера (500, 502, 503).

Ход работы:

Замеры выполнены на учебном API jsonplaceholder.typicode.com в браузере, запросы методов записи отправлены с заголовком Content-Type: application/json. Результаты приведены на рисунке 1, ниже разобраны по пунктам задания. Отдельно выполнена проверка того же адреса в браузере (рисунок 2) — ответ содержит 100 записей.

Практическая работа №1. Пользовательские запросы в веб-приложении

Результаты фактических HTTP-запросов к <https://jsonplaceholder.typicode.com>. Замеры выполнены 29.09.2026 23:54:43: код ответа, размер тела, время выполнения, число записей в JSON.

2.2 Первичный анализ запросов

Метод	URL	Код	Размер	Время, с	Записей	Content-Type
GET	/posts	200	26,9 КБ	0.875	100	application/json; charset=utf-8
GET	/posts?userId=1	200	2,7 КБ	0.722	10	application/json; charset=utf-8
GET	/posts/1	200	292 Б	0.679	1	application/json; charset=utf-8

2.3 Детальный разбор запроса /posts

Параметр	Значение
Метод	GET
URL	https://jsonplaceholder.typicode.com/posts
Параметры	отсутствуют
Код ответа	200 OK
Размер ответа	26,9 КБ
Время выполнения	0.875 с
Записей в ответе	100
Начало тела ответа	[{ "userId": 1, "id": 1, "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit", "body": "quia et suscipit\nsu...

Для сравнения — одиночная запись /posts/1: код 200, 292 Б, 0.679 с. Ответ меньше в 94 раз.

2.7 Ключевые заголовки ответа

Заголовок	Значение
content-type	application/json; charset=utf-8
cache-control	max-age=43200
etag	W/"6b80-Ybsq/K6GwwqrYkAsFxqDXGc7DoM"
date	Tue, 29 Sep 2026 20:54:43 GMT
server	cloudflare
content-encoding	gzip
vary	Origin, Accept-Encoding
x-powered-by	Express
expires	-1

Preview и Response. Вкладка Preview показывает ответ, разобранный браузером в структуру — по нему удобно проверить поля и вложенность. Вкладка Response показывает сырой текст ответа — по ней видно фактический формат, порядок полей и служебные символы. Для чтения данных удобнее Preview, для проверки формата и отладки — Response.

2.4–2.5 Как параметры уменьшают объём данных

Запрос	Записей	Размер	Время, с	Меньше /posts
/posts — все записи	100	26,9 КБ	0.875	1,0×
/posts?userId=1 — фильтр по пользователю	10	2,7 КБ	0.722	10,1×
/posts/1 — одна запись	1	292 Б	0.679	94,2×

Передача параметра `userId` сокращает ответ в 10 раз (100 записей против 10), а запрос одной записи — в 94 раз. Это основной приём снижения нагрузки на сеть: не запрашивать данные, которые не нужны.

2.6 Массовые запросы

Запрос	Код	Размер	Время, с
/posts/1	200	292 Б	0.937
/posts/2	200	278 Б	0.896
/posts/3	200	283 Б	0.652
Три запроса параллельно			0.937

Три запроса отправлены одновременно. Суммарное время 0.937 с — примерно столько же, сколько занимает один запрос, то есть браузер выполняет их параллельно, а не по очереди.

2.9 Методы HTTP: чтение и запись

Метод	URL	Код	Размер	Время, с	Тело ответа
POST	/posts	201	107 Б	0.952	{ "title": "Пост через Postman", "body": "Текст поста", "userId": 2, "id": 101 }
PUT	/posts/1	200	111 Б	0.954	{ "id": 1, "title": "Полное обновление", "body": "Новый текст", "userId": 1 }
PATCH	/posts/1	200	264 Б	1.301	{ "userId": 1, "id": 1, "title": "Изменён только заголовок", "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit
DELETE	/posts/1	200	2 Б	1.331	{}
HEAD	/posts/1	200	0 Б	0.689	
OPTIONS	/posts/1	204	0 Б	0.866	

Итоги по методам. GET — чтение, параметры в URL, данные не меняются. POST вернул 201 Created: ресурс создан, сервер присвоил новый идентификатор (101). PUT и PATCH вернули 200 OK. DELETE — 200 OK с пустым телом {}. HEAD — 200 OK без тела, только заголовки (так проверяют наличие и размер ресурса). OPTIONS — 204 No Content, применяется для предварительного запроса preflight.

Чем PUT отличается от PATCH. PUT заменяет ресурс целиком — клиент обязан передать все поля объекта. PATCH изменяет только переданные поля, остальные сохраняются. Для смены одного поля используют PATCH, чтобы не перезаписывать объект и не потерять данные.

Рекомендации по оптимизации запросов

- 1. Пагинация: запрашивать данные порциями, а не все 100 записей сразу.
- 2. Фильтрация на сервере: передавать `userId` и другие параметры — ответ меньше в 10 раз.
- 3. Запрос конкретной записи вместо списка, когда нужен один объект.
- 4. Кэширование: сервер отдаёт `Cache-Control: max-age=43200` и `ETag` — повторный запрос можно заменить проверкой `If-None-Match` и получить 304 без тела.
- 5. Сжатие: ответ приходит с `Content-Encoding: gzip`; без него 27 КБ превращаются в заметно больший объём.
- 6. Сокращение числа запросов: объединять независимые обращения и не дублировать вызовы.
- 7. Убирать лишние поля из ответа, если API это позволяет — лишние данные тоже передаются по сети.

DevTools или Postman. DevTools нужен, чтобы увидеть реальные запросы страницы: сколько их, в каком порядке, с какими заголовками, что взято из кэша и где потеряно время. Postman нужен, чтобы конструировать запросы вручную, работать с авторизацией и токенами, сохранять коллекции и писать автотесты. Первый инструмент наблюдает, второй — создаёт.

Рис. 1 – Результаты замеров: коды ответа, размер, время и число записей

```
Автоформатировать ☐
[
  {
    "userId": 1,
    "id": 1,
    "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
    "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
  },
  {
    "userId": 1,
    "id": 2,
    "title": "qui est esse",
    "body": "est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis\nqui aperiam non debitis possimus qui neque nisi nulla"
  },
  {
    "userId": 1,
    "id": 3,
    "title": "ea molestias quasi exercitationem repellat qui ipsa sit aut",
    "body": "ullam et saepe reiciendis voluptatem adipisci\nsit amet autem assumenda provident rerum culpa\nquis hic commodi nesciunt rem tenetur doloremque ipsam iure\nquis sunt voluptatem rerum illo velit"
  },
  {
    "userId": 1,
    "id": 4,
    "title": "eum et est occaecati",
    "body": "ullam et saepe reiciendis voluptatem adipisci\nsit amet autem assumenda provident rerum culpa\nquis hic commodi nesciunt rem tenetur doloremque ipsam iure\nquis sunt voluptatem rerum illo velit"
  },
  {
    "userId": 1,
    "id": 5,
    "title": "nesciunt quas odio",
    "body": "repudiandae veniam quaerat sunt sed\nnalias aut fugiat sit autem sed est\nvoluptatem omnis possimus esse voluptatibus quis\nest aut tenetur dolor neque"
  },
  {
    "userId": 1,
    "id": 6,
    "title": "dolorem eum magni eos aperiam quia",
    "body": "ut aspernatur corporis harum nihil quis provident sequi\nmollitia nobis aliquid molestiae\nperspiciatis et ea nemo ab reprehenderit accusantium quas\nvoluptate dolores velit et doloremque molestiae"
  },
  {
    "userId": 1,
    "id": 7,
    "title": "magnam facilis autem",
    "body": "dolore placeat quibusdam ea quo vitae\nmagni quis enim qui quis quo nemo aut saepe\nquidem repellat excepturi ut quia\nsunt ut sequi eos ea sed quas"
  }
]
```

Рис. 2 – Ответ /posts в браузере: массив из 100 объектов JSON

2.2 Первичный анализ запросов

Метод	URL	Код	Размер	Время, с	Записей
GET	/posts	200	26,9 КБ	0,875	100
GET	/posts?userId=1	200	2,7 КБ	0,722	10
GET	/posts/1	200	292 Б	0,679	1

2.3 Детальный разбор запроса /posts

Выбран запрос GET /posts. Параметров он не имеет, отвечает кодом 200, объём ответа 26,9 КБ, время выполнения 0,875 секунды, в ответе 100 записей. Для сравнения взят запрос /posts/1: тот же код ответа, но объём 292 байта — в 94 раза меньше, и это единственная запись. Вывод: адрес ресурса определяет объём данных, поэтому запрашивать нужно именно тот ресурс, который требуется, а не список целиком.

2.4 Эксперименты с запросами в консоли

Выполнение запроса через fetch() даёт тот же результат, что и переход по адресу в браузере: запрос виден в панели Network, ответ разбирается в объект. Сравнение /posts и /posts/1 показывает разницу в объёме (26,9 КБ против 292 байт) и в количестве записей (100 против 1); время выполнения отличается незначительно, так как определяется задержкой сети, а не объёмом данных при таких размерах.

2.5 Работа с параметрами

Запрос	Записей	Размер	Меньше /posts
/posts	100	26,9 КБ	—
/posts?userId=1	10	2,7 КБ	в 10,1 раза
/posts/1	1	292 Б	в 94,2 раза

Передача параметра `userId` сокращает ответ примерно в 10 раз. Это основной приём снижения нагрузки на сеть: сервер отдаёт только те записи, которые нужны, и остальные данные даже не передаются.

2.6 Массовые запросы

Три запроса `/posts/1`, `/posts/2` и `/posts/3` отправлены одновременно. Суммарное время составило 0,937 секунды — примерно столько же, сколько занимает один запрос. Это значит, что браузер выполняет запросы параллельно, а не последовательно. Избыточных запросов не обнаружено, размеры ответов сопоставимы (292, 278 и 283 байта).

2.7 Заголовки и ответы

Заголовок	Значение	Назначение
<code>content-type</code>	<code>application/json; charset=utf-8</code>	тип данных и кодировка
<code>cache-control</code>	<code>max-age=43200</code>	ответ можно кэшировать 12 часов
<code>etag</code>	<code>W/"6b80-..."</code>	версия ресурса для проверки изменений
<code>date</code>	дата ответа сервера	момент формирования ответа
<code>server</code>	<code>cloudflare</code>	сервер или CDN, отдавший ответ
<code>content-encoding</code>	<code>gzip</code>	ответ сжат при передаче
<code>vary</code>	<code>Origin, Accept-Encoding</code>	кэш зависит от источника и кодировки
<code>x-powered-by</code>	<code>Express</code>	используемый серверный фреймворк

Вкладки `Preview` и `Response` отличаются представлением. `Preview` показывает ответ, разобранный браузером в структуру, — по нему удобно проверять поля и вложенность. `Response` показывает сырой текст ответа — по нему видно фактический формат, порядок полей и служебные символы. Для чтения данных удобнее `Preview`, для проверки формата и отладки — `Response`.

2.8 Оптимизация запросов

1. Пагинация: запрашивать данные порциями вместо всех 100 записей сразу.
2. Фильтрация на сервере: параметр `userId` уменьшает ответ в 10 раз.
3. Запрос конкретной записи вместо списка, когда нужен один объект.
4. Кэширование: сервер отдаёт `Cache-Control: max-age=43200` и `ETag` — повторный запрос можно заменить проверкой `If-None-Match` и получить ответ 304 без тела.
5. Сжатие: ответ приходит с `Content-Encoding: gzip`; без сжатия 26,9 КБ заняли бы заметно больше.
6. Сокращение числа запросов: объединять независимые обращения и не дублировать вызовы.

7. Уменьшение объёма ответа: убирать лишние поля, если API это позволяет.

2.9 Работа с API через Postman

Через Postman (и эквивалентно — из браузера) выполнены все основные методы.

Результаты:

Метод	URL	Код	Размер	Что вернул
GET	/posts	200	26,9 КБ	массив из 100 объектов
GET	/posts/1	200	292 Б	один объект
POST	/posts	201	107 Б	созданный объект с присвоенным id 101
PUT	/posts/1	200	111 Б	объект, заменённый целиком
PATCH	/posts/1	200	264 Б	объект с изменённым только одним полем
DELETE	/posts/1	200	2 Б	пустой объект {}
HEAD	/posts/1	200	0 Б	только заголовки, без тела
OPTIONS	/posts/1	204	0 Б	без тела: какие методы поддерживаются

POST вернул 201 Created — ресурс создан, сервер присвоил новый идентификатор. PUT и PATCH вернули 200 OK, DELETE — 200 OK с пустым телом. PUT заменяет ресурс целиком, поэтому клиент обязан передать все поля; PATCH изменяет только переданные поля, остальные сохраняются. Для смены одного поля используют PATCH, чтобы не перезаписывать объект и не потерять данные.

2.10 Сравнение DevTools и Postman

DevTools	Postman
анализ реальных запросов страницы	ручное конструирование запросов
проверка производительности и кэширования	сохранение коллекций и повторное использование
работа в контексте открытого сайта: видны все запросы и их порядок	автотесты и скрипты
видно, что пришло из кэша, а что загружено заново	работа с авторизацией и токенами

Вывод: DevTools нужен, чтобы наблюдать за реальными запросами страницы и находить узкие места по времени и объёму. Postman нужен, чтобы создавать запросы вручную, отлаживать API, хранить коллекции и проверять поведение методов — то есть там, где запрос нужно именно построить, а не посмотреть.

Ответы на контрольные вопросы:

1. DevTools — встроенные инструменты разработчика. Elements показывает структуру документа, Console — ошибки и выполнение кода, Network — запросы, коды ответа, заголовки и время.

2. DevTools открывается клавишей F12. Для замеров производительности важны отключение кэша и настройка ограничения скорости в панели Network.

3. Части HTTP-запроса: метод (что делать), адрес с параметрами (с каким ресурсом), заголовки (служебная информация), тело (данные при создании и изменении).

4. GET — чтение, POST — создание, PUT — полная замена, PATCH — частичное изменение, DELETE — удаление. Примеры приведены в таблице методов выше.

5. HEAD запрашивает только заголовки — так проверяют наличие и размер ресурса. OPTIONS сообщает, какие методы поддерживает ресурс; его же отправляет браузер перед «непростым» запросом (preflight) при кросс-доменном обращении.

6. 1xx информационные, 2xx успешные (200 OK, 201 Created, 204 No Content), 3xx перенаправления (301, 302, 304 Not Modified), 4xx ошибки клиента (404), 5xx ошибки сервера (500).

7. REST — стиль взаимодействия с ресурсами по HTTP. Ресурс представлен уникальным адресом, например /posts/1 — конкретная запись.

8. JSON поддерживает объекты, массивы, строки, числа, логические значения и null. Пример объекта: {"userId": 1, "id": 1, "title": "sunt aut facere"}; пример массива: [{"id": 1}, {"id": 2}].

9. Preview показывает ответ, разобранный в структуру, Response — сырой текст. Первый удобен для чтения данных, второй — для проверки формата.

10. При оптимизации чаще всего смотрят Content-Type, Cache-Control, ETag, Authorization и Content-Encoding.

11. Приёмы снижения нагрузки: пагинация, фильтрация, кэширование, запрос конкретной записи вместо списка, сжатие ответа.

12. В Postman запрос создают вручную и сохраняют вместе с тестами, а в DevTools наблюдают запросы, которые браузер уже отправил. Первый нужен для разработки и отладки API, второй — для диагностики работы приложения.

Вывод:

Изучены пользовательские запросы веб-приложения и инструменты работы с ними. На учебном API выполнены запросы всеми основными методами: GET, POST, PUT, PATCH, DELETE, а также HEAD и OPTIONS, зафиксированы коды ответов, размеры тел и время выполнения. Установлено, что POST возвращает код 201 и присвоенный идентификатор, PUT и PATCH различаются полнотой передаваемых данных, а DELETE возвращает пустое тело.

Измерено, как параметры влияют на объём ответа: фильтрация по userId уменьшает ответ в 10 раз, запрос одной записи — в 94 раза. Проверено, что браузер выполняет несколько запросов параллельно: три одновременных обращения заняли столько же времени, сколько одно. Разобрано назначение ключевых заголовков ответа — Content-Type, Cache-Control, ETag, Content-Encoding, Vary.

Сформулированы меры оптимизации: пагинация, серверная фильтрация, запрос конкретной записи, кэширование с проверкой ETag, сжатие и сокращение числа запросов. Отдельно сопоставлены DevTools и Postman: первый служит для наблюдения за реальными запросами страницы, второй — для конструирования и отладки запросов вручную.

Примечание. Замеры значений и кодов ответов выполнены фактическими запросами к API; сводка приведена на рисунке 1. Снимки панели Network браузера и окон Postman в отчёт не включены: их автоматическое получение оказалось недоступным, поэтому при необходимости они делаются вручную — панель Network открывается клавишей F12 на странице jsonplaceholder.typicode.com/posts, а те же запросы повторяются в Postman по адресам из таблиц выше.